

Usage Guide

This guide covers how to **use** the graph-db-engine tools: the command-line interface (`graphdb-cli`), the interactive agent REPL, and the terminal UI (`graphdb-tui`). For **examples** of specific capabilities (OCR, RAG, image generation, etc.), see [EXAMPLES.md](#). For the **Python SDK** API reference, see [python/SDK.md](#).

Table of Contents

1. [Quick Start](#)
 2. [CLI Reference \(`graphdb-cli` \)](#)
 - [Common Flags](#)
 - [Subcommands](#)
 3. [Interactive Agent REPL \(`agent` \)](#)
 - [Launch](#)
 - [Slash Commands](#)
 - [Skills](#)
 - [Tool-Call Trace](#)
 - [Ingesting Your Own Code](#)
 4. [Terminal UI \(`graphdb-tui` \)](#)
 5. [Model Selection](#)
 - [Default \(Offline Stub\)](#)
 - [Real Local GGUF Model](#)
 - [GPU Acceleration](#)
 6. [Engine Selection \(LLM vs. Needle\)](#)
 7. [Python SDK](#)
-

Quick Start

```
# Build the workspace (Rust crates: rust, model-hub, cli, tui)
cargo build --workspace

# Run the interactive agent (Claude-Code-style REPL)
cargo run -p graphdb-cli -- agent

# Ask a one-off question
cargo run -p graphdb-cli -- ask "what is the Needle engine"

# Launch the terminal UI (full-screen chat + RAG + extraction modes)
cargo run -p graphdb-tui

# Generate text from a prompt
cargo run -p graphdb-cli -- generate "Explain Rust ownership in one sentence"

# List all CLI subcommands
cargo run -p graphdb-cli -- --help
```

CLI Reference (`graphdb-cli`)

The `graphdb-cli` provides a command-line interface to all model-hub pipelines. Every subcommand runs with **offline stub models** by default (no weights, no network). Point `--model / --tokenizer` at real GGUF weights to run a quantized Llama instead.

Common Flags

These flags apply to **all** subcommands (place them **before** the subcommand name):

Flag	Description	Default
<code>--model <PATH></code>	Path to a GGUF text model file	None (uses offline stub)
<code>--tokenizer <PATH></code>	Path to the matching tokenizer.json	None (required if <code>--model</code> is set)
<code>--device <DEVICE></code>	Compute device: <code>cpu</code> , <code>cuda</code> , <code>metal</code>	<code>cpu</code>

Example:

```
cargo run -p graphdb-cli -- \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  --device cpu \
  generate "Explain Rust traits"
```

Subcommands

generate

Generate text from a prompt.

```
cargo run -p graphdb-cli -- generate "Write a haiku about Rust"
```

Flags:

- `--max-tokens <N>` — maximum new tokens (default: 256)
- `--temperature <T>` — sampling temperature, 0 = greedy (default: 0.7)

embed

Produce a text embedding vector.

```
cargo run -p graphdb-cli -- embed "the Rust programming language" --engine needle
```

Flags:

- `--engine <ENGINE>` — `llm` or `needle` (default: `needle`)

Prints the embedding dimension and a preview of the first 8 values.

rag

Retrieval-augmented generation over a small demo knowledge base.

```
cargo run -p graphdb-cli -- rag "what is Rust" --k 4 --engine needle
```

Flags:

- `--k <N>` — number of chunks to retrieve (default: 4)
- `--engine <ENGINE>` — embedding engine for re-ranking: `llm` or `needle` (default: `needle`)

Prints the final answer and the source chunks (file path + relevance score).

extract

Extract structured JSON from text according to a schema.

```
cargo run -p graphdb-cli -- extract "Alice works at OpenAI since 2021" \
  --schema entities --engine needle
```

Flags:

- `--schema <SCHEMA>` — a built-in name (`entities`) or an inline JSON schema string (default: `entities`)
- `--engine <ENGINE>` — extraction engine: `llm` or `needle` (default: `needle`)

Prints the extracted JSON.

ask

Run the agentic orchestrator (tool calling + answer composition) on the demo knowledge base.

```
cargo run -p graphdb-cli -- ask "list all documents" --engine needle
```

Flags:

- `--engine <ENGINE>` — tool-routing engine: `llm` or `needle` (default: `needle`)

Prints the selected tool and the final answer.

transcribe

Transcribe a mono f32 PCM raw audio file (stub unless a real Whisper model is loaded).

```
cargo run -p graphdb-cli -- transcribe /path/to/audio.raw
```

Expects little-endian f32 samples at 16 kHz.

agent

Launch the **interactive Claude-Code-style REPL**. See the [dedicated section below](#).

```
cargo run -p graphdb-cli -- agent
```

Flags:

- `--context-dir <DIR>` — auto-ingest `.rs` / `.md` / `.txt` files from this directory at startup

info

Print system and build information: version, device, text model backend, GPU features, registered tools.

```
cargo run -p graphdb-cli -- info
```

Interactive Agent REPL (`agent`)

The `agent` subcommand launches a **Claude-Code-style REPL** — an interactive terminal session where you can ask questions about a codebase grounded in the graph knowledge base. It features:

- **Diamond prompt (`◆`)** with readline editing (arrow keys, history, Ctrl-C/D)
- **Difficulty routing** (Simple / Moderate / Complex) — each task is classified and routed to the appropriate engine(s)

- **Skill selection** — reusable instruction bundles loaded from disk (`model-hub/examples/skills/*/SKILL.md`)
- **Visible tool-call traces** — shows `● tool_use` and `⌘ tool_result` for every turn
- **Slash commands** — `/help` , `/skills` , `/tools` , `/context` , `/ingest` , `/clear` , `/quit`

Launch

```
# Fully offline (stub LLM):
cargo run -p graphdb-cli -- agent

# Ingest your own source tree at startup:
cargo run -p graphdb-cli -- agent --context-dir ./model-hub/src

# With a real local GGUF model:
cargo run -p graphdb-cli -- \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  agent
```

Startup Banner

```
graphdb agent • Claude-Code-style REPL  |

model      stub-text
skills     api-comparator, code-explainer, codebase-navigator, refactor-planner
tools      search_knowledge_base, list_documents, get_document_chunks
context    5 documents / 5 chunks

Type /help for commands, /quit to exit.

◆
```

Slash Commands

Type any of these at the `◆` prompt:

Command	Effect
<code>/help</code>	Show the full command table + routing table
<code>/skills</code>	List loaded skills (name, description, trigger cues)
<code>/tools</code>	List available tools with their parameters
<code>/context</code>	Show how many documents and chunks are in the KB
<code>/ingest <path> [title]</code>	Read a file and add it to the knowledge base (<code>.rs</code> , <code>.md</code> , <code>.txt</code>)
<code>/clear</code>	Clear the terminal screen
<code>/quit</code> <code>/exit</code> <code>/q</code>	Exit the REPL

Skills

Skills are reusable instruction bundles stored as `model-hub/examples/skills/<name>/SKILL.md` . Each skill has:

- A **name** (e.g. `code-explainer`)
- A **description** (one-line summary)
- **Trigger cues** (comma-separated keywords like `"explain, how, why, describe"`)
- **Instructions** (the body of the `SKILL.md` , injected into the LLM prompt when the skill fires)

When you enter a task, the agent:

1. Classifies the difficulty (Simple / Moderate / Complex)
2. Selects the best-matching skill by counting cue hits
3. Injects that skill's instructions into the LLM prompt (if LLM composition is used)

Four skills ship by default:

Skill	Description	Trigger Cues
codebase-navigator	Locate files, list documents, fetch chunks	list, show, find, where, which file, locate, chunk, sections of, what documents
code-explainer	Explain how a piece of the codebase works	explain, how, why, what does, walk me through, describe, understand
refactor-planner	Plan an implementation or refactor	refactor, implement, design, add, change, migrate, re-write, introduce, build
api-comparator	Compare options or summarize trade-offs	compare, versus, vs, trade-off, tradeoff, difference, summarize, pros and cons

To **add a new skill**, drop a `SKILL.md` into `model-hub/examples/skills/<name>/` — no code changes needed. Format:

```
---
name: my-skill
description: One-line summary of what the skill does.
cues: keyword1, keyword2, keyword3
---
The instruction body that will be injected into the LLM prompt.
Keep it concise and directive (e.g., "Respond with a numbered list...").
```

Tool-Call Trace

Every turn prints a **visible trace** of the tool-calling loop (just like Claude Code):

```
◆ explain how the Orchestrator delegates work

→ COMPLEX reasoning/generation cue
+ skill code-explainer

● tool_use search_knowledge_base({"query":"explain how..."})
  ↳ tool_result [{"chunk":{"id":"/model-hub/src/pipeline/orchestrator.rs#0",...}}]
(120-char preview)

handled by stub-text route → search_knowledge_base → stub-text compose

[The Orchestrator is the main agent turn driver. It first selects a tool
via the configured engine (LLM or Needle), executes it against the graph,
then has the text model compose a final answer grounded in the tool result.
Engine selection is per-capability via PipelineConfig.]
```

Routing table:

Difficulty	Tool routed by	Answer composed by	Skill injected?
SIMPLE	Needle (deterministic)	Deterministic format (no LLM)	No
MODERATE	Needle	Local LLM	Yes
COMPLEX	Local LLM	Local LLM	Yes

Ingesting Your Own Code

At startup:

```
cargo run -p graphdb-cli -- agent --context-dir ./my-project/src
```

Every `.rs`, `.md`, and `.txt` file under `./my-project/src` (non-recursive) is ingested into the knowledge base before the REPL starts.

During a session:

```
◆ /ingest ./my-project/src/main.rs MyProject
✓ ingested MyProject → 12 chunks total
```

Now your own code is searchable and can be retrieved via the `search_knowledge_base` tool.

Persistent History

Command history is saved to `~/.graphdb_agent_history` and restored on the next run. Arrow keys (↑/↓) navigate it.

Terminal UI (`graphdb-tui`)

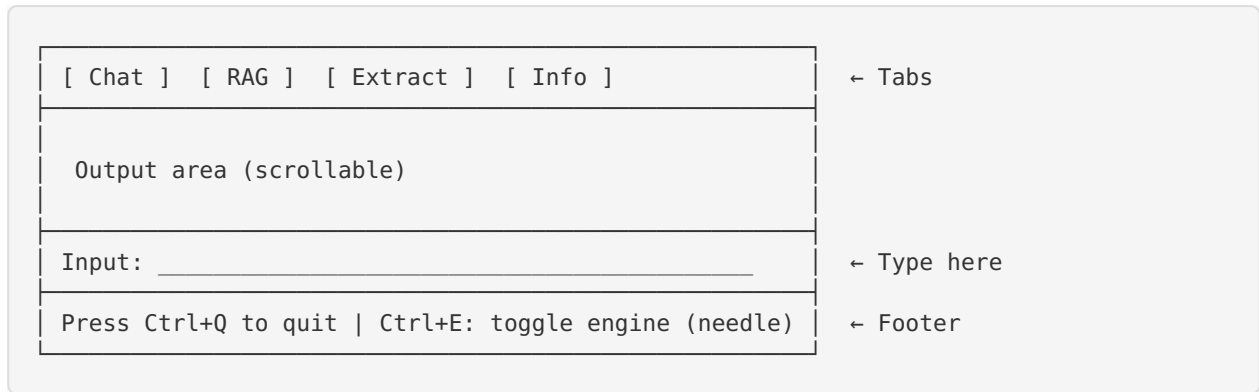
The `graphdb-tui` is a full-screen terminal UI for the model-hub pipelines, built with `ratatui` + `crossterm`. It provides four modes (tabs):

1. **Chat** — free-form text generation
2. **RAG** — retrieval-augmented generation over the demo knowledge base
3. **Extract** — structured extraction with the built-in `entities` schema
4. **Info** — system information (device, model backend, engine, tools)

Launch

```
cargo run -p graphdb-tui
```

Layout



Controls

Key	Action
Tab	Cycle through tabs (Chat → RAG → Extract → Info)
Enter	Submit the input
Ctrl+E	Toggle engine (llm ↔ needle)
Ctrl+Q / Esc	Quit

Modes

- **Chat:** Sends the input to `model.generate(...)`. The output is the generated text.
- **RAG:** Runs `RagPipeline::answer(...)` over the demo knowledge base. The output includes the answer and the source chunks.
- **Extract:** Runs `StructuredExtractor::extract(...)` with the built-in `entities` schema. The output is the extracted JSON.
- **Info:** Displays system information (device, model backend, engine, registered tools) — read-only, no input needed.

Model Selection

Default (Offline Stub)

By default, **every command** runs with an **offline stub model** (`StubTextModel`). This is a placeholder that:

- Generates boilerplate text (no real inference)
- Produces deterministic embeddings (no neural network)
- **Compiles and runs CPU-only with zero downloads**

Perfect for testing pipelines, CI, and examples.

Real Local GGUF Model

To run a **real quantized Llama model**, pass `--model` and `--tokenizer`:

```
cargo run -p graphdb-cli -- \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  generate "Explain Rust ownership"
```

The CLI will load the GGUF file via `candle-transformers` and run genuine inference. The `--model / --tokenizer` flags apply globally to **all** subcommands (including `agent`, `rag`, `ask`, etc.).

Where to get GGUF models:

- [Hugging Face Hub](#) — search for “GGUF” (e.g., `TheBloke/Llama-2-7B-GGUF`)
- Download a `.gguf` file and the matching `tokenizer.json`

GPU Acceleration

GPU backends are **off by default**. Enable them with cargo features:

```
# NVIDIA CUDA:
cargo run -p graphdb-cli --features cuda -- \
  --device cuda \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  generate "..."
```

```
# Apple Metal:
cargo run -p graphdb-cli --features metal -- \
  --device metal \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  generate "..."
```

Available features (per crate):

- `cuda` — NVIDIA GPU (via cuDNN)
- `metal` — Apple GPU (M1/M2/M3)
- `rocm` — AMD GPU (experimental)
- `mkl` — Intel MKL (CPU acceleration)
- `flash-attn` — Flash Attention 2 (CUDA only)

Without a feature flag, everything runs on CPU.

Engine Selection (LLM vs. Needle)

Many pipelines support **two engines** for tool calling, extraction, and embeddings:

Engine	Description	Speed	Accuracy
needle	Heuristic, regex-based, fully offline (no weights)	Instant	Good for deterministic tasks (list, lookup, simple entities)
llm	Neural text model (stub or real GGUF Llama)	Slower	Better for ambiguous queries, reasoning, complex extraction

Where to set it:

Command	Flag	Default
embed	--engine <ENGINE>	needle
rag	--engine <ENGINE>	needle
extract	--engine <ENGINE>	needle
ask	--engine <ENGINE>	needle
agent	(automatic routing by difficulty)	N/A

Example:

```
# Use Needle for fast deterministic extraction:
cargo run -p graphdb-cli -- extract "Alice at OpenAI" --engine needle

# Use the LLM for more nuanced extraction:
cargo run -p graphdb-cli -- \
  --model /path/to/model.gguf \
  --tokenizer /path/to/tokenizer.json \
  extract "Alice at OpenAI" --engine llm
```

In the **agent REPL**, engine selection is **automatic** — the difficulty router picks the cheapest engine that can handle the task:

- **Simple** → Needle only (no LLM)
- **Moderate** → Needle routes, LLM composes
- **Complex** → LLM routes + composes

You can toggle the default engine in the **TUI** by pressing **Ctrl+E**.

Python SDK

For Python users, the `ai_memory` package provides:

- `GraphStore` — the core graph storage (PyO3 wrapper around the Rust engine)
- `DatasetBuilder` — ingest documents and generate training datasets

- `NeedleAgentGroup` + `NeedleOrchestrator` — embedded function-calling agents (Needle2 integration)
- `PyCommunityDetector`, `PyGraphRAGRetriever`, `PyJEPAGraphRAG` — community detection, GraphRAG, and the JEPA world model

See [python/SDK.md](#) for the full API reference and [python/README.md](#) for setup instructions.

Further Reading

- [EXAMPLES.md](#) — Runnable examples for each capability (GraphBackend, JEPA, OCR, image generation, transcription, web scraping, segmentation, Claude Code agent)
 - [README.md](#) — Project overview, architecture, and build instructions
 - [python/SDK.md](#) — Python API reference
 - [model-hub/examples/](#) — Example source code (`.rs` files)
 - [model-hub/examples/skills/](#) — Skill definitions (`.md` files)
-

Tip: Run any command with `--help` to see its full usage:

```
cargo run -p graphdb-cli -- --help
cargo run -p graphdb-cli -- generate --help
cargo run -p graphdb-cli -- agent --help
```